NLP.

# Experiment - 04.

- **Aim:** Create a transformer from scratch using the Pytorch library.

- **Theory:**

(1) **Introduction -**

- Transformers are a deep learning architecture introduced in the paper by (Vaswani et. al, 2017). Unlike traditional sequence models such as RNNs and LSTMs, transformers rely entirely on the attention mechanism to process sequential data.
- This allows parallel computation, improved long range dependency handling, and state of the art performance in Natural language processing (NLP), computer vision, and multimodal tasks.

- Creating a transformer from scratch using Pytorch helps understand the internal components such as embeddings, positional encoding, multi head attention, feed forward layers, and encoder decoder stacks.

(2) **Objective -**

- To understand the architecture of a transformer model.
- To implement core components using Pytorch.

- To build encoder and decoder blocks manually.
- To train the model on sequence data for tasks such as translation or text generation.

## (3). Transformer Architecture overview -

- A transformer consists of two main parts :
  1. Encoder.
  2. Decoder.
- Each encoder and decoder contains stacked layers with attention and feed forward networks.
- Main components -
  - Input Embedding.
  - Positional Encoding.
  - Multi head Self attention.
  - feed forward network.
  - Layer normalization
  - Residual Connections.
  - Output linear + Softmax.

## (4). Mathematical Foundations -

### 4.1. Scaled dot Product Attention -

The attention mechanism computes relationships between tokens using query (Q) key (K) and value (V).

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Where,

- $Q$ → Query Matrix
- $K$ → Key Matrix
- $V$ → Value Matrix
- $d_k$ → Dimension of Key Vectors.

The scaling factor prevents large dot product values that could push softmax into very small gradients.

## 4.2. Multihead Attention -

Instead of one attention, multiple attention heads learn different relationships.

$$\text{Multihead}(Q, K, V) = \text{Concat}(\text{head}_1, \cdots \text{head}_h)W^o$$

Each head :

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

## 4.3. Feed forward Network.

- Each token passes through a fully connected network.

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

- This introduces nonlinearity and feature transformation.

## (5). Implementation Steps in PyTorch -

Step 1: Embedding layer.
Convert tokens into dense vectors using nn. Embedding.

## Step 2: Positional Encoding

Since transformers do not have sequence awareness, positional encoding adds order information using sine and cosine functions.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

## Step 3: Self attention Module.

- Compute Q, K, V using linear layers.
- Apply scaled dot product attention.
- Apply masking if needed.

## Step 4: Multi Head attention.

Split embeddings into multiple heads, perform attention and concatenate outputs.

## Step 5: Encoder Block.

Each layer encoder includes:

- Multihead self attention.
- Residual connection + Layer Norm.
- Feed forward Network.
- Residual Connection + Layer Norm.

## Step 6: Decoder Block.

Decoder contains:

- Masked self attention.
- Encoder decoder attention.
- Feed forward Network.

Step 7: Stacking layers.
Multiple encoder and decoder layers are stacked to form the transformer.

Step 8: Output layer.
Final linear layer converts hidden representation into vocabulary probabilities using softmax.

(6) Advantages of Transformers -

- Parallel computation.
- Handles long dependencies better than RNNs.
- Highly scalable.
- Foundation for modern AI models ( BERT, GPT, T5, Vision Transformers).

(7) Applications -

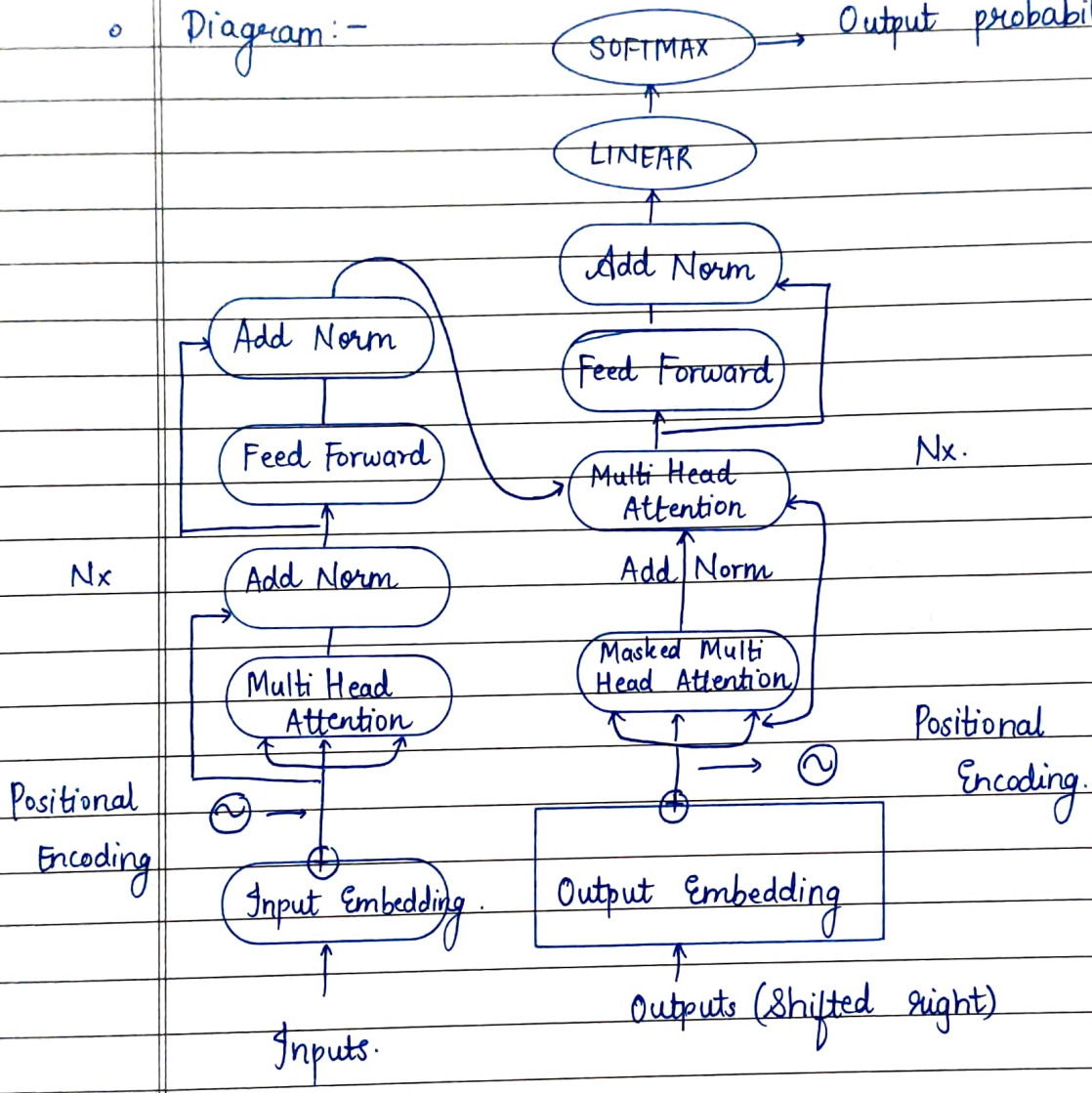
- Machine translation.
- Chatbots.
- Text summarization.
- Speech recognition
- deepfake detection, cyberbullying detection.
- Vision tasks.

(8) Challenges.

- High computational costs.
- Requires large datasets.

- Memory intensive.
- Training instability without proper tuning.

- Diagram:-




The transformer Model Architecture.

- CONCLUSION: Thus we successfully studied and implemented the transformer using PyTorch.

* * * * * * *.